

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ "ЛЬВІВСЬКА ПОЛІТЕХНІКА"



**АВТОМАТИЗОВАНЕ ПРОЕКТУВАННЯ КОМП'ЮТЕРНИХ
СИСТЕМ**

Лабораторна робота №3

Реалізація серверної та клієнтської частини гри.

Виконав: ст. гр. КІ-404
Стефанів В.В.
Прийняв:
Федак П.Р.

Львів 2024

Мета роботи: Розробити серверну та клієнтську частину гри відповідно до варіанту (камінь-ножиці-папір).

ТЕОРЕТИЧНІ ВІДОМОСТІ

Клієнт-серверна архітектура — це модель взаємодії між комп'ютерами, в якій клієнт (зазвичай це програма або пристрій користувача) надсилає запити до сервера, а сервер обробляє їх та повертає необхідну інформацію або виконує певну операцію.

Основні компоненти:

1. **Клієнт** — програма або пристрій, який ініціює запит. Клієнт надсилає серверу інформацію або запити на отримання певних даних. Прикладами можуть бути веб-браузери, мобільні додатки, або інші пристрої, які звертаються до сервера.
2. **Сервер** — це центральний обчислювальний вузол, який приймає запити від клієнтів, обробляє їх і відповідає результатами. Сервери можуть виконувати різні функції, такі як зберігання даних, обробка логіки бізнесу, управління автентифікацією та багато іншого.

Принцип роботи:

1. Клієнт надсилає запит до сервера (наприклад, HTTP-запит для завантаження веб-сторінки).
2. Сервер приймає запит, обробляє його (наприклад, витягує дані з бази даних або виконує бізнес-логіку).
3. Сервер повертає відповідь клієнту (наприклад, HTML-сторінку або JSON-дані).

Переваги клієнт-серверної архітектури:

- **Масштабованість** — сервери можуть обробляти багато клієнтів одночасно.
- **Централізація** — дані та логіка зберігаються на сервері, що спрощує управління.
- **Розподіл завдань** — клієнт відповідає за інтерфейс, а сервер — за зберігання даних і виконання логіки.

Приклади використання:

- Веб-додатки (сервери обслуговують браузері або мобільні додатки).
- Ігрові платформи з багатьма гравцями.
- Системи обміну файлами та повідомленнями.

Клієнт-серверна архітектура є фундаментальною концепцією для сучасних мережеских додатків, включаючи веб-сайти, мобільні додатки та хмарні сервіси.

ЗАВДАННЯ

1. Розробити серверну та клієнтську частину гри.
2. Обов'язкові кроки до кожної лабораторної.

Implement Server (HW) and Client (SW) parts of game (FEF)
1. Develop Server and Client. 2. Required steps.

Student number	Game	config format
20	rock paper scissors	JSON

Виконання роботи

1. Код для клієнта написаний мовою веб-програмування TypeScript з використанням бібліотек React (яку використав для побудови інтерфейсу), Electron (яку використав для отримання десктоп-програми з веб інтерфейсу), а також SerialPort (яку використав для роботи з серійними портами).
Оскільки програма написана для веб-інтерфейсу, то окрім окремої програми, нею можна користуватися прямо в браузері.

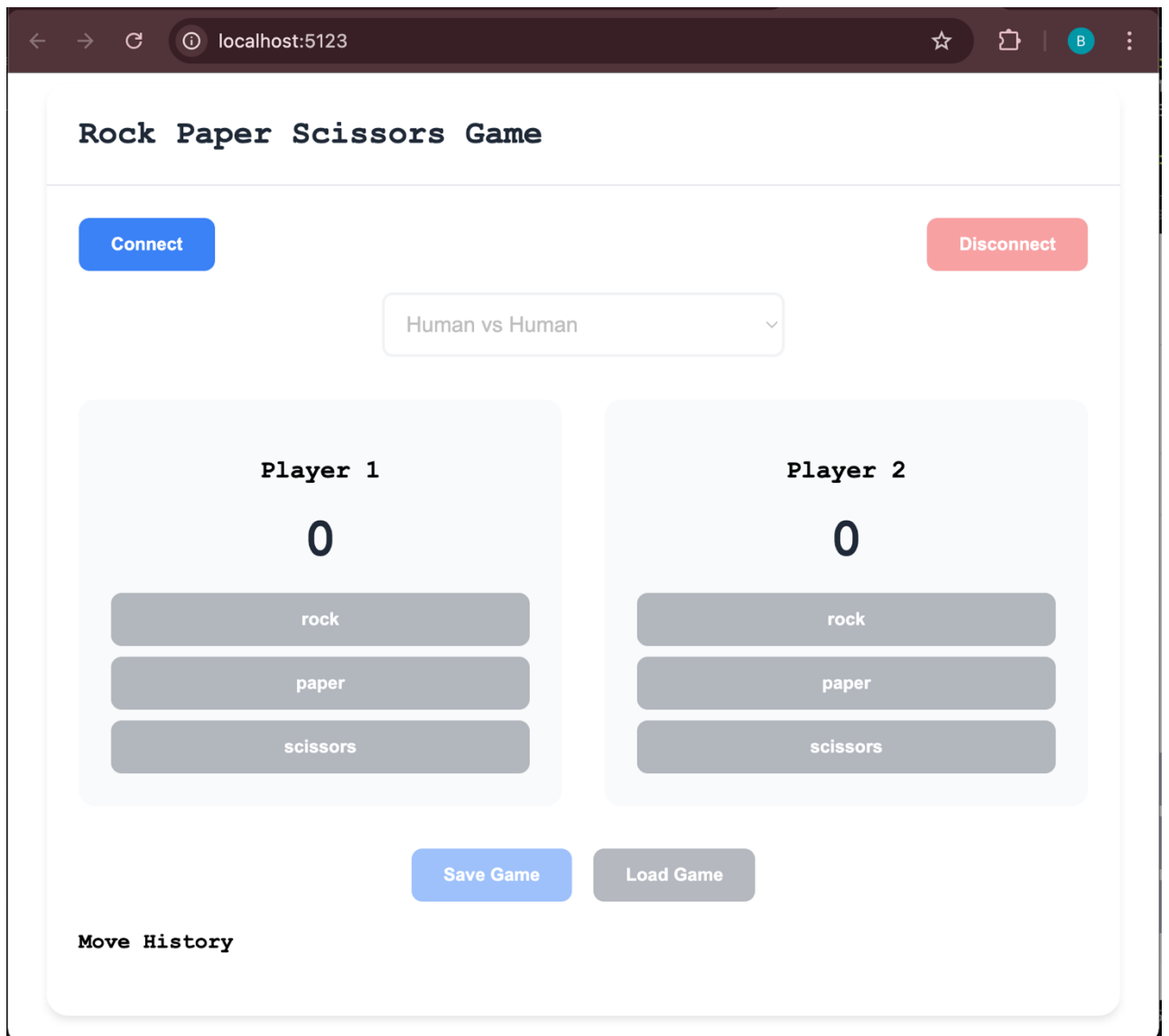


Рис.1. Клієнтська частина в браузері

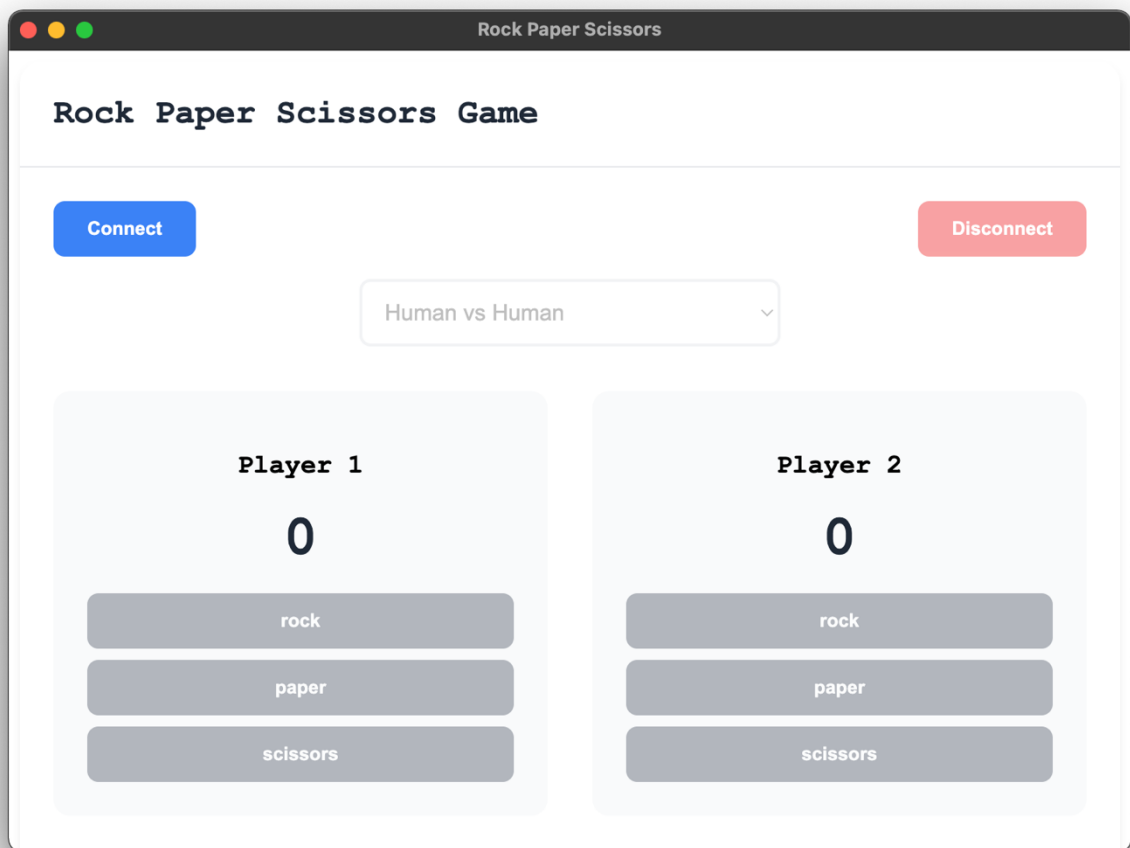


Рис.2. Клієнтська частина в десктоп-додатку

2. Написав код для серверу С-подібною мовою програмування Arduino, для якої він і написаний.

```
server > * server.ino
288 void loop() {
289   if (Serial.available() > 0) {
290     String input = Serial.readStringUntil('\n');
291     StaticJsonDocument<200> doc;
292     deserializeJson(doc, input);
293
294     String command = doc["command"];
295     GameManager* game = GameManager::getInstance();
296
297     if (command == "move") {
298       String move = doc["move"];
299       bool isPlayer1 = doc["isPlayer1"];
300       String response = game->processMove(move, isPlayer1);
301       Serial.println(response);
302     }
303     else if (command == "setMode") {
304       String mode = doc["mode"];
305       String response = game->setGameMode(mode);
306       Serial.println(response);
307     }
308     else if (command == "save") {
309       String savedState = game->saveGame();
310       Serial.println(savedState);
311     }
312     else if (command == "load") {
313       String savedState = doc["state"];
314       game->loadGame(savedState);
315       Serial.println("{\"status\":\"loaded\"}");

```

Рис.3. Приклад коду серверної частини

4. В ході виконання роботи було використано декілька патернів проектування, таких як Observer, Composite, Strategy, Facade, Factory і т.д. Всі патерни можна переглянути в лістингу коду проекту, вони відзначені коментарями.

5. Запустимо програму та перевіримо правильність виконання коду цілої системи. Протестував всі режими гри а також збереження та завантаження

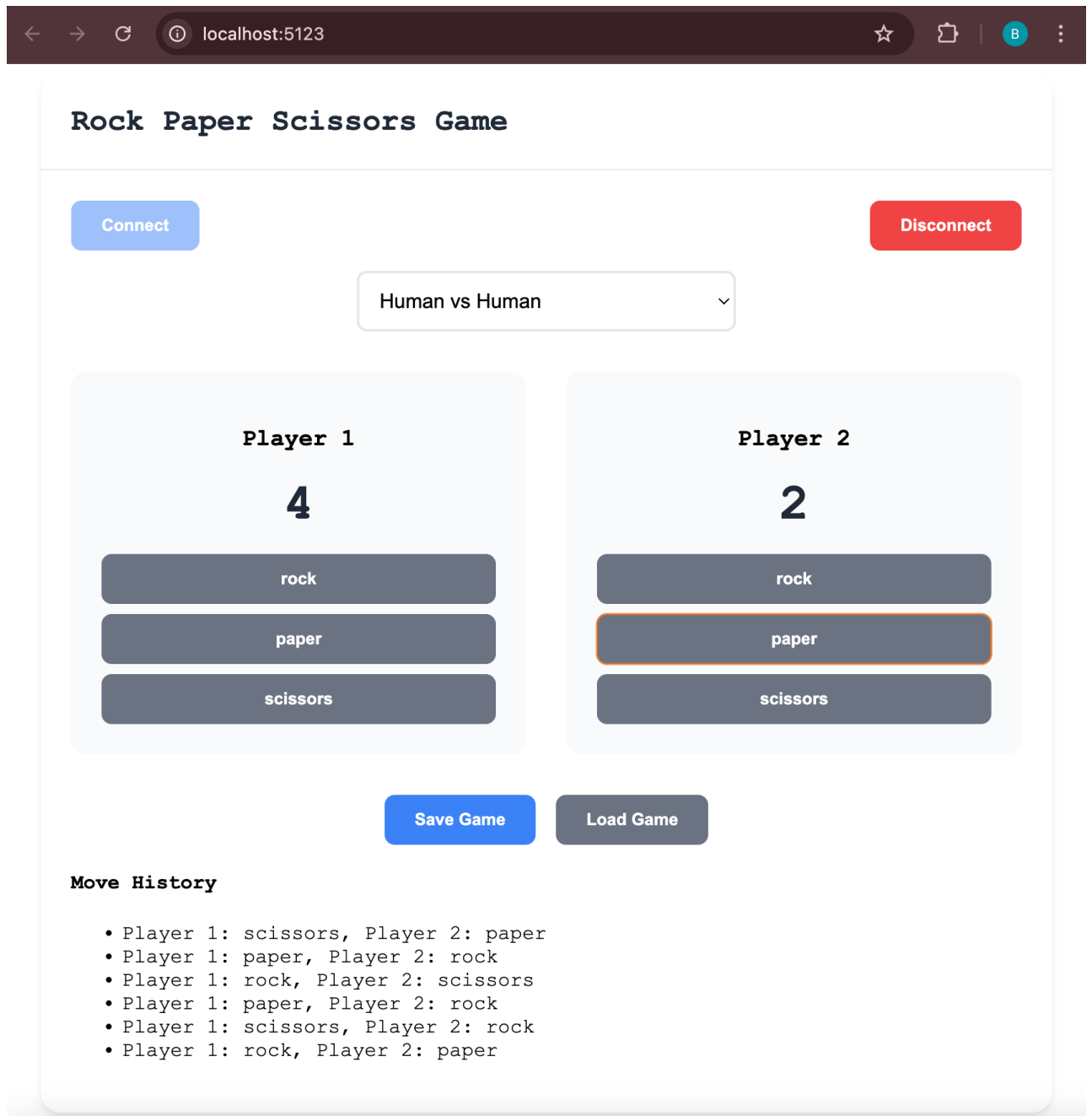


Рис.4. Гра людини проти людини

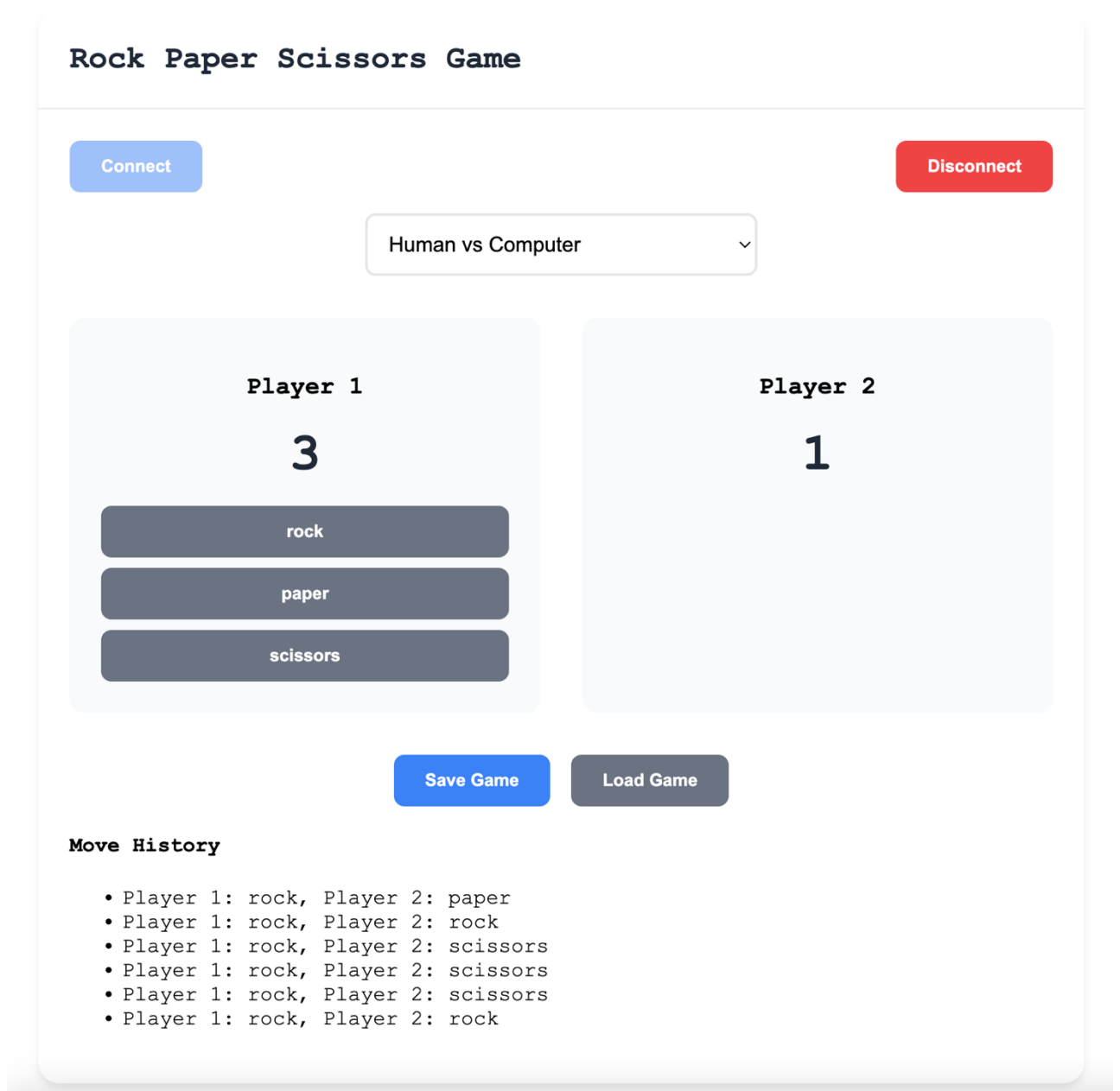


Рис.5. Гра людини проти комп'ютера

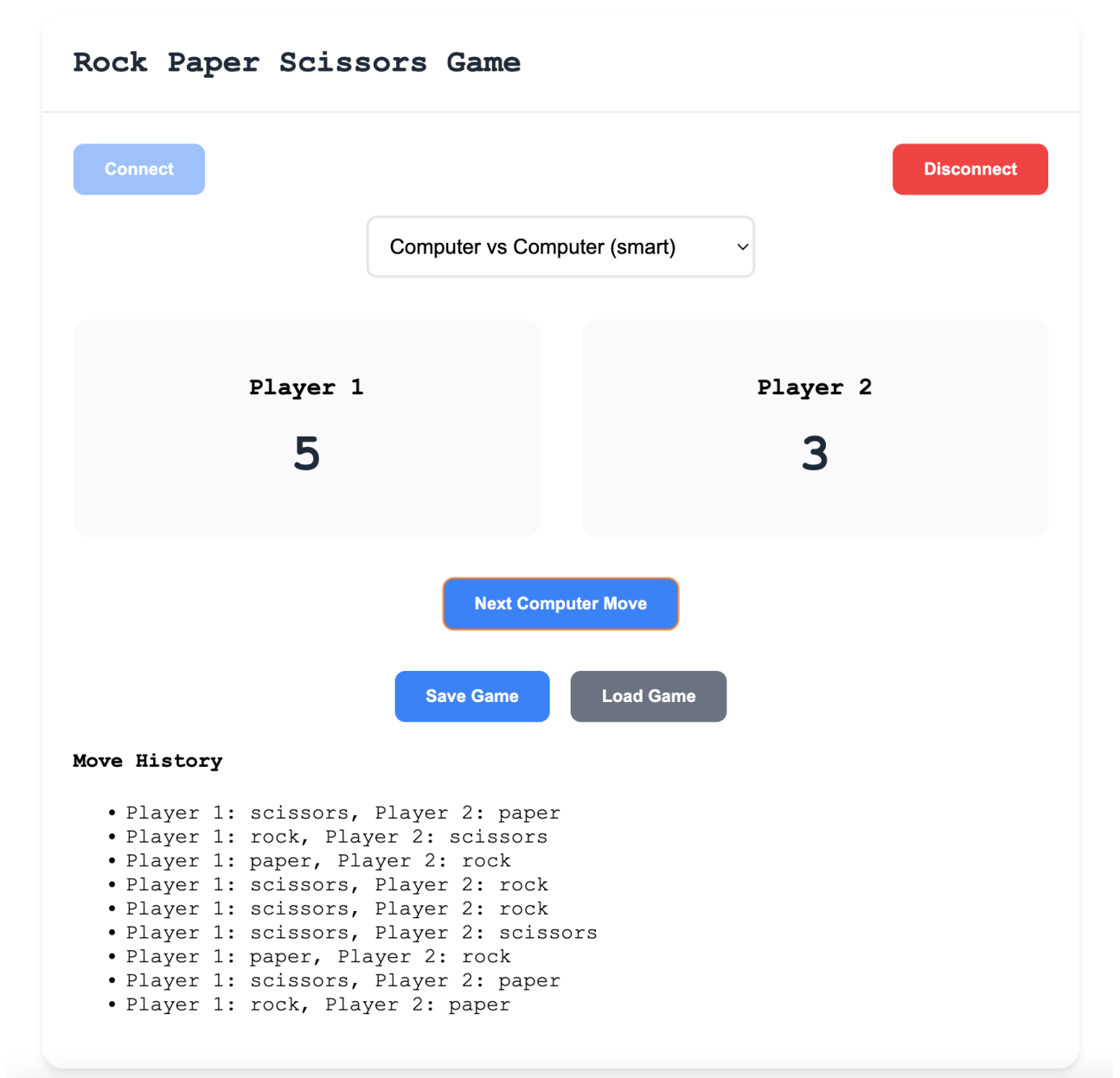


Рис.6. Гра комп'ютера проти комп'ютера

6. Зміни всі були створені на новій гілці. Весь код є в репозиторії. Окрім цього, він є наведений в додатках.

Висновок: У ході виконання лабораторної роботи я налаштував комунікацію та обмін повідомленнями між апаратним сервером і програмним клієнтом за допомогою UART. Для розробки серверної частини використав Arduino, для клієнтської TypeScript, React, Electron. Перевінив правильність виконання з допомогою юніт-тестів, написаних з Vitest та Jest, а також експериментально. Налаштував CI для збірки, тестування та створення артефактів, а також зробив необхідні зміни до репозиторію.

Додаток

Лістинг коду клієнтської частини:

```
import React from "react";
import styles from "../styles.module.css";

// Паттерн Composite для створення складених компонентів
interface CardProps {
  children: React.ReactNode;
  className?: string;
}

export const Card: React.FC<CardProps> = ({ children, className = "" }) => (
  <div className={` ${styles.card} ${className}`}>{children}</div>
);

export const CardHeader: React.FC<CardProps> = ({
  children,
  className = "",
}) => <div className={` ${styles.cardHeader} ${className}`}>{children}</div>;

export const CardTitle: React.FC<CardProps> = ({
  children,
  className = "",
}) => <h2 className={` ${styles.cardTitle} ${className}`}>{children}</h2>;

export const CardContent: React.FC<CardProps> = ({
  children,
  className = "",
}) => <div className={` ${styles.cardContent} ${className}`}>{children}</div>;

import React, { useEffect, useState } from "react";
import { useSerialCommunication, useGameState } from "../../hooks/hooks";
import {
  Button,
  Select,
  Card,
  CardHeader,
```

```

    CardTitle,
    CardContent,
  } from "../components";
import { GameMode, Move, SerialMessage, GameState } from "../../types/types";
import styles from "./styles.module.css";

const INITIAL_STATE: GameState = {
  player1Score: 0,
  player2Score: 0,
  gameMode: "hvh",
  lastMove1: "",
  lastMove2: "",
};

const MODE_OPTIONS = [
  { value: "hvh" as GameMode, label: "Human vs Human" },
  { value: "hvc" as GameMode, label: "Human vs Computer" },
  { value: "cvc" as GameMode, label: "Computer vs Computer" },
  { value: "cvcs" as GameMode, label: "Computer vs Computer (smart)" },
];

const MOVES: Move[] = ["rock", "paper", "scissors"];

const RockPaperScissors: React.FC = () => {
  const { connect, disconnect, sendMessage, receivedMessage, isConnected } =
    useSerialCommunication();
  const { state, updateState } = useGameState(INITIAL_STATE);
  const [moveHistory, setMoveHistory] = useState<string[]>([]);

  useEffect(() => {
    if (receivedMessage) {
      console.log(receivedMessage);
      try {
        const response = JSON.parse(receivedMessage);

        if (receivedMessage.includes("loaded")) {
          updateState(JSON.parse(localStorage.getItem("state") ?? ""));
          return;
        }
      } catch {
        // ignore
      }
    }
  }, [receivedMessage]);

```

```

    } else if (receivedMessage.includes("saved")) {
        localStorage.setItem("state", JSON.stringify(response));
        return;
    }
    updateState(response);
    if (response.move1 && response.move2) {
        setMoveHistory((prev) => [
            `Player 1: ${response.move1}, Player 2: ${response.move2}`,
            ...prev,
        ]);
    }
} catch (e) {
    console.error("Failed to parse game response:", e);
}
}, [receivedMessage, updateState]));

```

```

const handleMove = (move: Move, isPlayer1: boolean) => {
    const message: SerialMessage = {
        command: "move",
        move,
        isPlayer1,
    };
    sendMessage(JSON.stringify(message));
};

```

```

const handleModeChange = (mode: GameMode) => {
    const message: SerialMessage = {
        command: "setMode",
        mode,
    };
    sendMessage(JSON.stringify(message));
    setMoveHistory([]);
};

```

```

const handleSaveGame = () => {
    const message: SerialMessage = {
        command: "save",
    };
    sendMessage(JSON.stringify(message));
};

```

```

    };
    sendMessage(JSON.stringify(message));
  };

const handleLoadGame = () => {
  const message: SerialMessage = {
    command: "load",
    state: JSON.parse(localStorage.getItem("state") ?? ""),
  };
  sendMessage(JSON.stringify(message));
};

const handleNextComputerMove = () => {
  const message: SerialMessage = {
    command: "move",
    isPlayer1: true,
  };
  sendMessage(JSON.stringify(message));
};

return (
  <Card className={styles.container}>
    <CardHeader>
      <CardTitle>Rock Paper Scissors Game</CardTitle>
    </CardHeader>
    <CardContent>
      <div className={styles.controls}>
        <Button
          onClick={() => connect()}
          disabled={isConnected}
          variant="primary"
        >
          Connect
        </Button>
        <Button
          onClick={() => disconnect()}
          disabled={!isConnected}
          variant="danger"

```

```

    >
      Disconnect
    </Button>
  </div>

  <div className={styles.modeSelect}>
    <Select
      options={MODE_OPTIONS}
      value={state.gameMode}
      onChange={handleModeChange}
      disabled={!isConnected}
    />
  </div>

  <div className={styles.players}>
    <div className={styles.player}>
      <h3>Player 1</h3>
      <p className={styles.score}>{state.player1Score}</p>
      {state.gameMode !== "cvc" && state.gameMode !== "cvcs" && (
        <div className={styles.moves}>
          {MOVES.map((move) => (
            <Button
              key={move}
              onClick={() => handleMove(move, true)}
              disabled={!isConnected}
              variant="secondary"
            >
              {move}
            </Button>
          ))}
        </div>
      )}
    </div>

    <div className={styles.player}>
      <h3>Player 2</h3>
      <p className={styles.score}>{state.player2Score}</p>
      {state.gameMode === "hvh" && (

```

```

    <div className={styles.moves}>
      {MOVES.map((move) => (
        <Button
          key={move}
          onClick={() => handleMove(move, false)}
          disabled={!isConnected}
          variant="secondary"
        >
          {move}
        </Button>
      ))}
    </div>
  )}
</div>
</div>

{(state.gameMode === "cvc" || state.gameMode === "cvcs") && (
  <div className={styles.computerControls}>
    <Button onClick={handleNextComputerMove} disabled={!isConnected}>
      Next Computer Move
    </Button>
  </div>
)}

<div className={styles.gameActions}>
  <Button
    onClick={handleSaveGame}
    disabled={!isConnected}
    variant="primary"
  >
    Save Game
  </Button>
  <Button
    onClick={() => handleLoadGame()}
    disabled={!isConnected}
    variant="secondary"
  >
    Load Game

```

```

        </Button>
    </div>

    <div className={styles.moveHistory}>
        <h4>Move History</h4>
        <ul>
            {moveHistory.map((move, index) => (
                <li key={index}>{move}</li>
            ))}
        </ul>
    </div>
</CardContent>
</Card>
);
};

export { RockPaperScissors };

import styles from "./styles.module.css";

interface SelectProps<T extends string | number> {
    options: { value: T; label: string }[];
    onChange: (value: T) => void;
    value: T;
    disabled?: boolean;
    className?: string;
    name?: string;
    id?: string;
}

export function Select<T extends string | number>({
    options,
    onChange,
    value,
    className = "",
    ...props
}: SelectProps<T>) {
    return (

```

```

<select
  className={` ${styles.select} ${className}`}
  value={value}
  onChange={(e) => {
    const newValue = (
      typeof value === "number" ? Number(e.target.value) : e.target.value
    ) as T;
    onChange(newValue);
  }}
  {...props}
>
  {options.map((option) => (
    <option key={String(option.value)} value={option.value}>
      {option.label}
    </option>
  ))}
</select>
);
}

```

```

import React, { createContext, useState, useEffect } from "react";
import { SerialPort } from "serialport";

```

```

interface SerialContextType {
  connect: () => Promise<void>;
  disconnect: () => void;
  sendMessage: (message: string) => void;
  receivedMessage: string | null;
  isConnected: boolean;
}

```

```

const SerialContext = createContext<SerialContextType | undefined>(undefined);

```

```

const SerialContextProvider: React.FC<{ children: React.ReactNode }> = ({
  children,
}) => {
  const [port, setPort] = useState<SerialPort | null>(null);
  const [writer, setWriter] = useState<WritableStreamDefaultWriter | null>(

```

```

    null
  );
  const [reader, setReader] = useState<ReadableStreamDefaultReader | null>(
    null
  );
  const [receivedMessage, setReceivedMessage] = useState<string | null>(null);
  const [, setMessageBuffer] = useState<string>("");
  const [isConnected, setIsConnected] = useState<boolean>(false);

  const connect = async () => {
    try {
      const selectedPort = await (
        navigator as unknown as { serial: any }
      ).serial.requestPort();
      await selectedPort.open({ baudRate: 9600 });
      setPort(selectedPort);
      setIsConnected(true);

      const textEncoder = new TextEncoderStream();
      textEncoder.readable.pipeTo(selectedPort.writable);
      setWriter(textEncoder.writable.getWriter());

      const textDecoder = new TextDecoderStream();
      selectedPort.readable.pipeTo(textDecoder.writable);
      const reader = textDecoder.readable.getReader();
      setReader(reader);
    } catch (error) {
      console.error("Failed to connect:", error);
    }
  };

  const disconnect = async () => {
    try {
      if (reader) {
        reader.releaseLock();
      }
      if (writer) {
        writer.releaseLock();
      }
    }
  };

```

```

    }
    if (port) {
        port.close();
    }
} catch (err) {
    console.log(err);
} finally {
    setIsConnected(false);
}
};

```

```

const sendMessage = async (message: string) => {
    if (writer) {
        await writer.write(message + "\n");
    } else {
        console.error("Writer is not available");
    }
};

```

```

useEffect(() => {
    if (!reader) return;

```

```

    const readLoop = async () => {
        while (true) {
            const { value, done } = await reader.read();
            if (done) break;
            if (value) {
                setMessageBuffer((prev) => {
                    const newBuffer = prev + value;
                    const endIdx = newBuffer.indexOf("\n");
                    if (endIdx !== -1) {
                        const fullMessage = newBuffer.slice(0, endIdx);
                        setReceivedMessage(fullMessage);
                        return newBuffer.slice(endIdx + 1);
                    }
                    return newBuffer;
                });
            }
        }
    };
}

```

```

    }
  };

  readLoop().catch((err) => console.error(err));

  return () => {
    reader.cancel();
  };
}, [reader]);

return (
  <SerialContext.Provider
    value={{ connect, disconnect, sendMessage, receivedMessage, isConnected }}
  >
    {children}
  </SerialContext.Provider>
);
};

export { SerialContextProvider, type SerialContextType, SerialContext };

export {
  SerialContextProvider,
  type SerialContextType,
  SerialContext,
} from "../serial-context-provider/serial-context-provider";

export { RockPaperScissors } from "../rock-paper-scissors/rock-paper-scissors";
export { Button } from "../button/button";
export { Select } from "../select/select";
export { Card, CardHeader, CardTitle, CardContent } from "../card/card";

export { useSerialCommunication } from "../use-serial-communication.hook";
export { useGameState } from "../use-game-state.hook";

import { useState, useCallback } from "react";
import { GameState } from "../types/types";

```

```

// Паттерн Observer реалізований через хук
export const useGameState = (initialState: GameState) => {
  const [state, setState] = useState<GameState>(initialState);

  const updateState = useCallback((newState: Partial<GameState>) => {
    setState((prev) => ({ ...prev, ...newState }));
  }, []);

  return { state, updateState };
};

import { useContext } from "react";
import { SerialContext, SerialContextType } from "../components/components";

const useSerialCommunication = (): SerialContextType => {
  const context = useContext(SerialContext);
  if (!context) {
    throw new Error("useSerial must be used within a SerialProvider");
  }
  return context;
};

export { useSerialCommunication };

export type GameMode = "hvh" | "hvc" | "cvc" | "cvcs";
export type Move = "rock" | "paper" | "scissors";

export interface GameState {
  player1Score: number;
  player2Score: number;
  gameMode: GameMode;
  lastMove1: Move | "";
  lastMove2: Move | "";
}

export interface SerialMessage {
  command: "move" | "setMode" | "save" | "load";
  move?: Move;
}

```

```

    isPlayer1?: boolean;
    mode?: GameMode;
    state?: GameState;
}

import { StrictMode } from "react";
import { SerialContextProvider } from "../libs/components/components";
import { RockPaperScissors } from "../libs/components/components";
import { createRoot } from "react-dom/client";

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <SerialContextProvider>
      <RockPaperScissors />
    </SerialContextProvider>
  </StrictMode>
);

```

Лістинг коду серверної частини:

```

#include <ArduinoJson.h>

// Паттерн State для збереження стану гри
class GameState {
public:
    int player1Score = 0;
    int player2Score = 0;
    String gameMode = "hvh"; // hvh, hvc, cvc, cvcs
    String lastMove1 = "";
    String lastMove2 = "";
    String lastComputerMove = "";
};

// Паттерн Strategy для різних режимів гри
class GameStrategy {
public:
    virtual String makeMove() = 0;
    virtual void updateLastMove(String opponentMove) {}
};

```

```

class RandomStrategy : public GameStrategy {
public:
    String makeMove() {
        String moves[] = {"rock", "paper", "scissors"};
        return moves[random(0, 3)];
    }
};

class SmartStrategy : public GameStrategy {
private:
    String lastOpponentMove = "";
    int rockCount = 0;
    int paperCount = 0;
    int scissorsCount = 0;

public:
    String makeMove() {
        if (lastOpponentMove == "") {
            return RandomStrategy().makeMove();
        }

        int maxCount = max(max(rockCount, paperCount), scissorsCount);

        if (lastOpponentMove == "rock") {
            return "paper";
        } else if (lastOpponentMove == "paper") {
            return "scissors";
        } else {
            return "rock";
        }
    }

    void updateLastMove(String opponentMove) override {
        lastOpponentMove = opponentMove;
        if (opponentMove == "rock") rockCount++;
        else if (opponentMove == "paper") paperCount++;
        else if (opponentMove == "scissors") scissorsCount++;
    }
}

```

```
};
```

```
// Паттерн Singleton для керування грою
```

```
class GameManager {
```

```
    private:
```

```
        static GameManager* instance;
```

```
        GameState state;
```

```
        GameStrategy* computerStrategy1;
```

```
        GameStrategy* computerStrategy2;
```

```
    GameManager() {
```

```
        computerStrategy1 = new RandomStrategy();
```

```
        computerStrategy2 = new SmartStrategy();
```

```
    }
```

```
    public:
```

```
        static GameManager* getInstance() {
```

```
            if (!instance) {
```

```
                instance = new GameManager();
```

```
            }
```

```
            return instance;
```

```
        }
```

```
String processMove(String playerMove, bool isPlayer1) {
```

```
    StaticJsonDocument<200> doc;
```

```
    if (state.gameMode == "cvcs") {
```

```
        String move1 = computerStrategy1->makeMove();
```

```
        String move2 = computerStrategy2->makeMove();
```

```
        updateScore(move1, move2);
```

```
        computerStrategy2->updateLastMove(move1);
```

```
        doc["move1"] = move1;
```

```
        doc["move2"] = move2;
```

```
    } else if (state.gameMode == "cvc") {
```

```
        String move1 = computerStrategy1->makeMove();
```

```
        String move2 = computerStrategy1->makeMove();
```

```

        updateScore(move1, move2);

        doc["move1"] = move1;
        doc["move2"] = move2;
    } else if (state.gameMode == "hvc") {
        String computerMove = computerStrategy1->makeMove();
        updateScore(playerMove, computerMove);

        doc["move1"] = playerMove;
        doc["move2"] = computerMove;
    } else {
        if (isPlayer1) {
            state.lastMove1 = playerMove;
        } else {
            state.lastMove2 = playerMove;
            updateScore(state.lastMove1, state.lastMove2);
        }

        doc["move1"] = state.lastMove1;
        doc["move2"] = state.lastMove2;
    }

    doc["player1Score"] = state.player1Score;
    doc["player2Score"] = state.player2Score;
    doc["gameMode"] = state.gameMode;

    String output;
    serializeJson(doc, output);

    if (!isPlayer1) {
        state.lastMove1 = "";
        state.lastMove2 = "";
    }

    return output;
}

String setGameMode(String mode) {

```

```

    StaticJsonDocument<200> doc;
    state.gameMode = mode;
    resetScores();
    doc["player1Score"] = state.player1Score;
    doc["player2Score"] = state.player2Score;
    doc["gameMode"] = state.gameMode;

    String output;
    serializeJson(doc, output);
    return output;
}

```

```

void resetScores() {
    state.player1Score = 0;
    state.player2Score = 0;
    state.lastMove1 = "";
    state.lastMove2 = "";
    state.lastComputerMove = "";
}

```

```

String saveGame() {
    StaticJsonDocument<200> doc;
    doc["status"] = "saved";
    doc["player1Score"] = state.player1Score;
    doc["player2Score"] = state.player2Score;
    doc["gameMode"] = state.gameMode;

    String output;
    serializeJson(doc, output);
    return output;
}

```

```

void loadGame(String savedState) {
    StaticJsonDocument<200> doc;
    deserializeJson(doc, savedState);

    state.player1Score = doc["player1Score"];
    state.player2Score = doc["player2Score"];
}

```

```

        state.gameMode = doc["gameMode"].as<String>();
    }

private:
    void updateScore(String move1, String move2) {
        if (move1 == move2) return;

        if ((move1 == "rock" && move2 == "scissors") ||
            (move1 == "paper" && move2 == "rock") ||
            (move1 == "scissors" && move2 == "paper")) {
            state.player1Score++;
        } else if (move1 != "") {
            state.player2Score++;
        }
    }
}

};

GameManager* GameManager::instance = nullptr;

void setup() {
    Serial.begin(9600);
    randomSeed(analogRead(0));
}

void loop() {
    if (Serial.available() > 0) {
        String input = Serial.readStringUntil('\n');
        StaticJsonDocument<200> doc;
        deserializeJson(doc, input);

        String command = doc["command"];
        GameManager* game = GameManager::getInstance();

        if (command == "move") {
            String move = doc["move"];
            bool isPlayer1 = doc["isPlayer1"];
            String response = game->processMove(move, isPlayer1);
            Serial.println(response);
        }
    }
}

```

```
}  
else if (command == "setMode") {  
    String mode = doc["mode"];  
    String response = game->setGameMode(mode);  
    Serial.println(response);  
}  
else if (command == "save") {  
    String savedState = game->saveGame();  
    Serial.println(savedState);  
}  
else if (command == "load") {  
    String savedState = doc["state"];  
    game->loadGame(savedState);  
    Serial.println("{\"status\":\"loaded\"}");  
}  
}  
}
```